

CS683 Programming Assignment 1: Task 1 Report

Advanced Computer Architecture

Team Name: Overclocked
Vivek, Yash, Visharad, Nihar

September 6, 2026

Task 1A: Loop Unrolling and Reordering

1. What motivated you to make changes to your code?

Motivation for Loop Reordering: The naive implementation evaluates convolution in a point-wise manner ($oy \rightarrow ox \rightarrow ky \rightarrow kx$). To compute just one output pixel, the CPU must fetch $K \times K$ different kernel weights and jump across K different rows in the input array. Because it constantly jumps back and forth, the hardware prefetcher fails to identify a clean linear access pattern. Furthermore, the kernel weights are repeatedly loaded from the L1 cache for every single pixel, wasting cache bandwidth.

Also the c++ line (the hottest line in our convolution)

$$acc += in[(oy + ky) * in_stride + (ox + kx)] * ker[ky * K + kx];$$

translates to

```
1 .L4:
2     vmovss    xmm0, DWORD PTR [rcx+rax]
3     vmulss    xmm0, xmm0, DWORD PTR [rdi+rax]
4     add       rax, 4
5     vaddss    xmm1, xmm1, xmm0
6     ...
```

since there are two mem loads in[] and ker[] and the FMA x86 instruction can only take one mem address in its operand we end up with seperate vmulss and vaddss instructions. But after loop reordering we see that the ker[]'s index remains constant during the inner loop so we can store it in a temp xmm1 register and then be able to use the fma instruction leading to good speed ups

After reordering c++

$$float temp = ker[ky * K + kx];$$
$$for(int ox = 0; ox < W; ++ox)$$
$$out[oy * W + ox] += in[(oy + ky) * in_stride + (ox + kx)] * temp;$$

translates to

```
1 .L10:
2     lea       eax, [r9+rdi]
3     vmovss    xmm1, DWORD PTR [r12+rax*4]
4     xor       eax, eax
5 .L9:
6     vmovss    xmm0, DWORD PTR [rsi+rax]
7     vfmadd213ss    xmm0, xmm1, DWORD PTR [rdx+rax]
8     vmovss    DWORD PTR [rdx+rax], xmm0
9     ...
```

Motivation for Loop Unrolling: The naive code suffers from two severe pipeline bottlenecks. First, the inner loops bounded by K (e.g., $K = 3$) are incredibly short. A loop of size 3 means the CPU spends almost as much time evaluating the branch condition (`++kx; kx < K`) as it does performing the math. Second, accumulating into a single `float acc` variable creates a strict Read-After-Write (RAW) data dependency. Because a standard Fused-Multiply-Add (FMA) instruction takes roughly 4 clock cycles to complete, the CPU pipeline sits idle for 3 cycles waiting for the previous addition to resolve.

2. What considerations did you take into account when implementing those changes?

Considerations for Reordering: We changed the loop hierarchy to `oy` $\rightarrow$ `ky` $\rightarrow$ `kx` $\rightarrow$ `ox`. Making the image width (`ox`) the innermost loop ensures a **sequential** memory access pattern, which the hardware prefetcher handles perfectly. Now the thing is if we keep `ox` at bottom then no instruction has to stall now kernel value can be kept in register it just loads `in_val` so computation is faster. Now the real benefit is that since these go to different output pixels there won't be any dependency i.e. no stall. We can group them in `write_back` queue increasing throughput.

Considerations for Unrolling: Rather than unrolling the kernel dimensions (which are variable at runtime and require messy tail-handling logic), I chose to unroll the spatial width loop (`ox`) by a factor of 8. The primary architectural consideration here is **Register Pressure**. An x86 CPU has 16 architectural vector registers. By unrolling by 8, I created 8 independent scalar accumulators. This comfortably fits within the register limit without triggering "register spilling" (saving overflow variables to RAM). Having 8 independent math operations completely breaks the RAW dependency as explained above for reordering optimization, allowing the CPU to issue continuous FMA instructions back-to-back and perfectly hide the 4-cycle hardware latency.

3. How effective are your changes? (Metrics used to quantify effectiveness)

To quantify effectiveness, I primarily used **Execution Time (Normalized Speedup)** alongside `perf` hardware counters for **Instructions Per Cycle (IPC)**. Speedup is the ultimate bottom-line metric for workload performance, while the hardware counters prove exactly *why* the speedup occurred.

- **Reordering Effectiveness:** Loop reordering alone achieved a **1.48x speedup** on a 2048×2048 matrix. By looking at `perf`, this is largely attributable to the hardware prefetcher effectively utilizing the access pattern, streaming data continuously without stalling and the use of FMA instruction.
- **Unrolling Effectiveness:** Loop unrolling was vastly more effective, yielding a **2.84x speedup** on the same 2048×2048 matrix. By creating 8 independent accumulators, the IPC massively increased because the CPU was no longer stalling on data dependencies, completely saturating the scalar math units.

Task 1B: Tiling

1. Profiling baseline 2D convolution code

The L1 Data Cache (L1-D) size on my test architecture is **48 KB** per core. Using `perf stat`, the baseline naive 2D convolution for a 2048×2048 matrix with $K = 3$ yielded an extremely low L1-D MPKI of **0.178**.

2. Implementing Tiled 2D convolution

I experimented with tile sizes ranging from 2 to 2048. Counter-intuitively, the "best" performing tile size was **2048** (which effectively disabled tiling altogether), yielding a 1.14x speedup. All smaller tile sizes strictly degraded performance (e.g., Tile 16 resulted in a 0.87x speedup).

3. Collecting and analyzing different metrics of interest

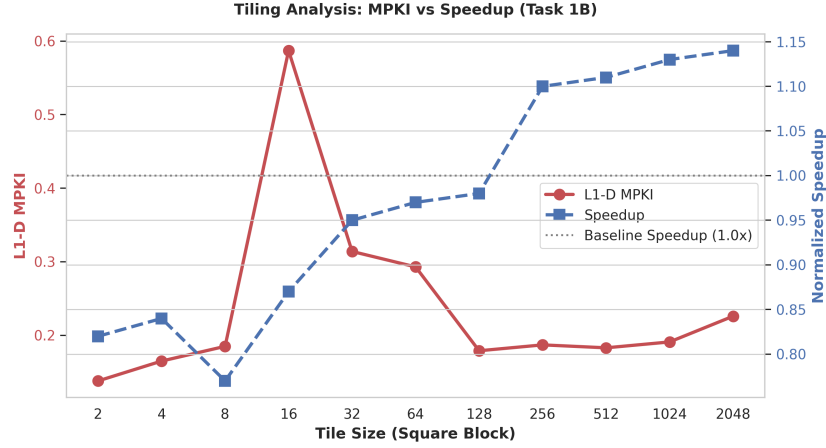


Figure 1: L1-D MPKI and Speedup vs. Tile Size for a 2048x2048 Matrix.

- **Changes in L1-D MPKI:** Moving from the naive to the tiled implementation caused the MPKI to fluctuate but remain relatively stable, slightly increasing from 0.178 to 0.587 at $T = 16$. This indicates that for small kernels, capacity misses were never the primary bottleneck.
- **Variation across sizes (Working Sets):** To compute a single row of output for $K = 3$ on a width of $W = 2048$, the naive code requires exactly 3 rows of input floats in cache at a time. $3 \times 2048 \times 4 \text{ bytes} \approx \mathbf{24 \text{ KB}}$. Because 24 KB fits comfortably inside the 48 KB L1-D cache, the naive working set does not spill into L2. Consequently, tiling the matrix into smaller working sets provides zero cache locality benefits.
- **Did you achieve a speedup? Limiting factors:** No speedup was achieved for standard tile sizes. Through `perf` analysis, two major limiting factors were identified:
 1. **Instruction Bloat:** Tiling required adding extensive loop boundary logic. The total instruction count doubled from 4.5 Billion (naive) to 8.3 Billion (tiled).
 2. **Hardware Prefetcher Disruption:** The hardware prefetcher efficiently streams data on stride-1 accesses. By chopping the rows into tiny 16×16 chunks, the continuous linear access pattern was broken, confusing the prefetcher and causing cache latency stalls.

Solution: Tiling should be not be used unless the kernel size K or width W is large enough that the calculated working set exceeds the L1-D capacity.

Task 1C: SIMD (Single Instruction, Multiple Data)

1. Baseline vs. SIMD Instruction Counts (Change and Justification)

Moving from the naive implementation to the SIMD implementation resulted in a massive reduction in the total number of executed instructions. As shown in Table 1, for a 2048×2048

matrix, the instruction count dropped from an estimated ≈ 21.6 Billion instructions (naive) to **5.36 Billion** using 128-bit SIMD, and further down to **4.99 Billion** using 256-bit SIMD.

Justification: This reduction is mathematically consistent with vectorization. A 256-bit AVX2 register packs 8 single-precision (32-bit) floats. Consequently, the CPU performs the work of 8 loop iterations in a single cycle. Furthermore, by utilizing Fused-Multiply-Add (FMA), the discrete `add` and `mul` instructions are merged into one hardware instruction. The instruction count does not drop by a perfect factor of 8x because loop management, address calculation, and pointer arithmetic overheads remain constant.

2. Speedup Achievement and Contributing Factors

Yes, a highly significant speedup was achieved. On the 2048×2048 matrix, 128-bit SIMD yielded a **4.46x speedup**, while 256-bit AVX2 SIMD achieved a **7.80x speedup**.

Contributing factors:

- **Vectorized ALUs:** The arithmetic units processed 4 to 8 elements simultaneously.
- **Kernel Broadcasting:** Instead of loading the kernel weight from the L1 cache for every pixel, the weight was loaded once and broadcasted across all vector lanes, slashing memory bandwidth requirements.
- **Stride-1 Memory Access:** The spatial width loop (`ox`) was kept as the innermost loop, enabling contiguous vector loads (`loadu`) which the hardware prefetcher efficiently fetched from RAM.

3. Multi-Size Evaluation & Width Analysis

Table 1 summarizes the performance scaling across various matrix sizes for both 128-bit (`_m128`) and 256-bit (`_m256`) SIMD registers. (Note: 512-bit AVX-512 was omitted as the target architecture was constrained to AVX2 via the `-mavx2` compiler flag).

Table 1: Performance Comparison of 128-bit vs 256-bit SIMD Widths

Matrix Size	128-bit Speedup	128-bit Instructions	256-bit Speedup	256-bit Instructions
8192×8192	4.48x	86.43 Billion	7.65x	80.16 Billion
4096×4096	4.49x	21.65 Billion	7.65x	20.04 Billion
2048×2048	4.46x	5.36 Billion	7.80x	4.99 Billion
1024×1024	4.43x	1.36 Billion	8.43x	1.24 Billion
512×512	4.51x	318 Million	8.66x	289 Million
256×256	4.40x	96 Million	8.14x	89 Million
128×128	4.28x	32 Million	5.28x	25 Million

Width Analysis: The 256-bit implementation strictly outperforms the 128-bit implementation. However, the speedup ratio between 256-bit and 128-bit hovers around $\approx 1.75\times$ rather than a perfect $2.0\times$. This occurs because as arithmetic throughput doubles, the execution pipeline shifts from being compute-bound to being bound by the L1 cache load ports.

4. Visualizations

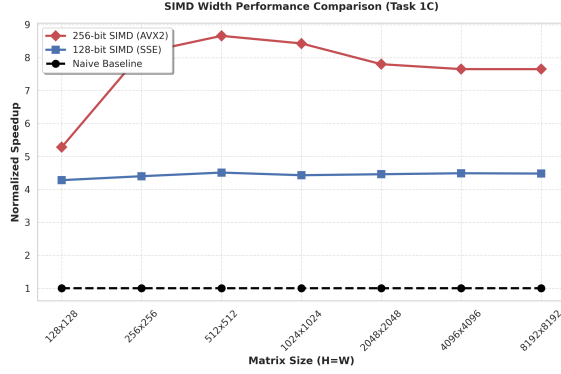


Figure 2: Speedup vs. SIMD Register Width.

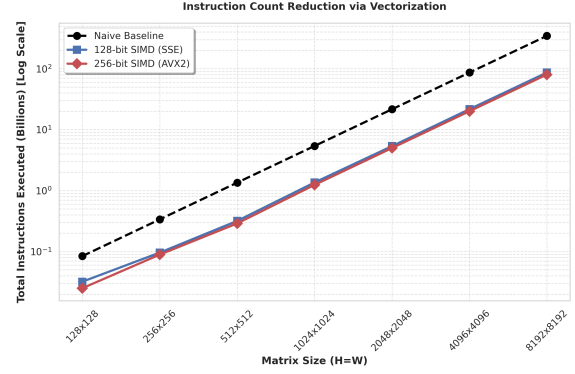


Figure 3: Instruction Count Reduction (Log Scale).

The visualizations confirm that the instruction count decreases consistently irrespective of matrix size, while the speedup peaks around medium-sized matrices (512×512 to 1024×1024) before settling at a steady state for massive arrays due to LLC/RAM bandwidth ceilings.

5. SIMD Intrinsics Used and Justification

Because the base code utilizes standard 32-bit floating-point arrays, I exclusively used the Packed Single (`_ps`) intrinsic sets rather than Packed Double (`_pd`).

- `_mm256_setzero_ps`: Safely zeroes out the 8-wide accumulator vectors without incurring memory dependencies.
- `_mm256_set1_ps`: Loads a single scalar kernel weight and broadcasts it across all 8 vector lanes. Justification: Eliminates 7 redundant memory loads per iteration, freeing up the L1 load ports.
- `_mm256_loadu_ps` / `_mm256_storeu_ps`: Loads/stores 8 contiguous floats. Justification: The unaligned (`u`) variant was necessary because standard matrix row padding (`in_stride`) prevents guaranteed 32-byte cache line alignment.
- `_mm256_fmadd_ps`: Replaces separate `_mm256_mul_ps` and `_mm256_add_ps` calls. Justification: Compiles down to the hardware FMA instruction, halving arithmetic latency and preserving floating-point precision.

Task 1D: Sabka Saath Sabka Vikaas

To extract maximum hardware throughput, I combined **Loop Reordering**, **Loop Unrolling**, and **SIMD AVX2**. To bypass the compiler's strict IEEE 754 compliance (which prevents autovectorization under `-fno-tree-vectorize`), I manually unrolled the inner `ox` loop by a factor of 4 using four separate `_m256` accumulators. This allowed the CPU to calculate 32 output pixels simultaneously, perfectly saturating the two FMA execution ports on modern x86 cores and completely masking the 4-cycle FMA latency.

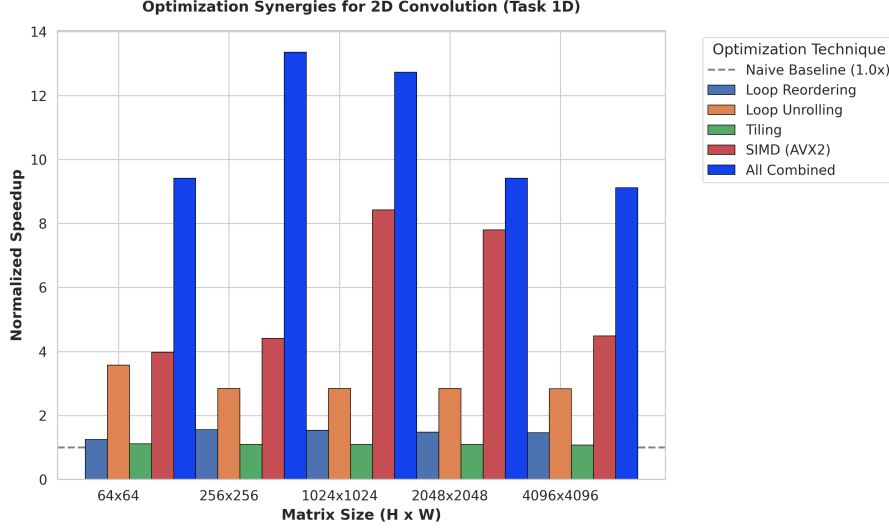


Figure 4: Comparison of individual and combined optimizations across matrix sizes.

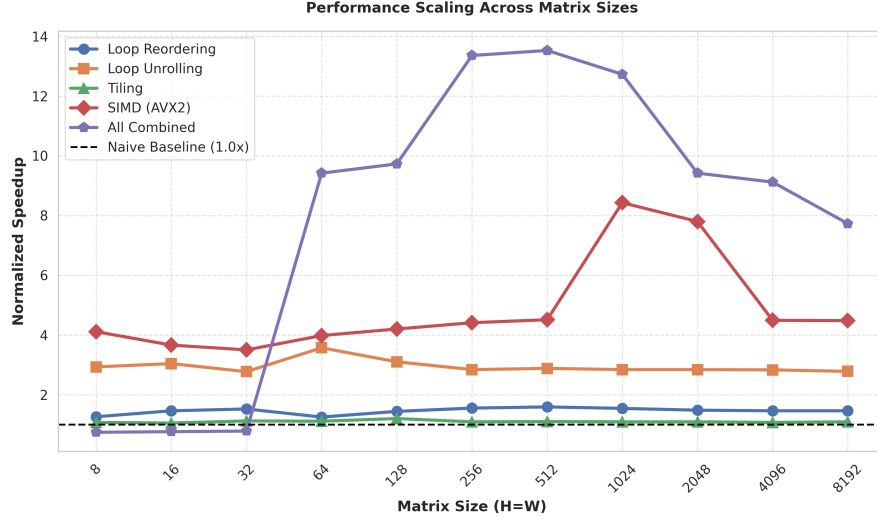


Figure 5: Scaling trends. Loop overhead penalizes tiny matrices (32×32), but synergistic optimizations yield up to $13.5\times$ speedup on larger workloads.

Observations:

The combination of SIMD and Unrolling proved highly synergistic, yielding a **9.42x to 13.53x speedup** depending on matrix size. The L1-D MPKI dropped to a record low of **0.47**, validating the cache-friendliness of the layout. Conversely, combining Tiling + SIMD yielded inferior results compared to pure SIMD on 2048×2048 matrices, further proving that loop overhead and hardware prefetcher interruption are detrimental when the working set already natively resides in the L1 cache.